

Ingeniería de LLMs

El Ecosistema Hugging Face y Código Abierto

Bienvenido al mundo del Open Source. En este dossier recopilamos todo el material práctico. Aprenderás a navegar por la plataforma de Hugging Face, configurar entornos con GPU en la nube usando Google Colab y ejecutar modelos de IA de vanguardia para texto, imagen y audio mediante *Pipelines*.

1. Conceptos Clave



Modelos (>1M)

El cerebro de la IA. Modelos como Llama 3, Mistral o FLUX, listos para descargar e integrar en tus proyectos.



Datasets (>200K)

Colecciones masivas de datos para afinar (Fine-Tuning) y entrenar a tus propios modelos de IA.



Spaces

Aplicaciones interactivas de la comunidad. Ejemplo: *Comic Factory*, *Outsmart* o *Kolors Virtual Try-On*.

2. Guía de Google Colab y Tokens

A. Generar tu Token en Hugging Face

1. Crea una cuenta en **huggingface.co**.
2. Ve a tu perfil > *Settings* > *Access Tokens*.
3. Crea un nuevo token con permisos de **Lectura (Read)** y cópialo (empieza por hf_...).

B. Activar la GPU y guardar Secretos en Colab

1. Crea un nuevo cuaderno en **Google Colab**.
2. Ve a **Entorno de ejecución > Cambiar tipo de entorno** y selecciona **T4 GPU**. (Obligatorio para que los modelos no den error de memoria).
3. Abre el panel lateral izquierdo y pulsa en el icono de la **Llave (Secretos)** .
4. Añade un secreto llamado HF_TOKEN, pega tu token y **activa el interruptor azul** (Acceso al cuaderno).

3. Práctica: Generación de Imágenes (FLUX.1)

FLUX.1-schnell es un modelo gigante (más de 30GB) que requiere optimizaciones especiales de memoria para funcionar en la versión gratuita de Colab. Completa el código con lo aprendido:

```
# 1. Instalar librerías e iniciar sesión
!pip install diffusers accelerate torch huggingface_hub
from google.colab import userdata
from huggingface_hub import login
login(userdata.get('HF_TOKEN'))

# 2. Cargar el modelo FLUX y optimizar memoria (Offload Secuencial)
import torch
from diffusers import FluxPipeline

pipe = FluxPipeline.from_pretrained("black-forest-labs/FLUX.1-schnell", torch_dtype=torch.bfloat16)
# TODO: Activa el offload secuencial de CPU aquí para no quedarte sin memoria
# TODO: Activa el slicing y el tiling del VAE aquí

generator = torch.Generator(device="cpu").manual_seed(0)

# 3. Definir el prompt y generar
prompt = "ESCRIBE_AQUI_TU_PROMPT_EN_INGLES"

# TODO: Llama a la variable 'pipe' pasándole el prompt, guidance_scale=0.0, num_inference_steps=4, max_sequence_length=512
image = ???

# TODO: Guarda la imagen resultante
???
```

4. El Gran Bazar de Pipelines

Los *Pipelines* son la API de alto nivel de Hugging Face. Nos permiten realizar inferencia de inteligencia artificial con apenas un par de líneas de código.

Preparación: Crea una celda exclusiva al principio de tu Colab para instalar todo lo necesario:

```
!pip install transformers datasets soundfile sentencepiece sacremoses
```

4.1 Análisis de Sentimiento

Evalúa un texto para determinar si su tono emocional es positivo o negativo.

Tarea: "sentiment-analysis"

Modelo Sugerido: (Déjalo en blanco para usar el de defecto)

```
from transformers import pipeline

# TODO: Crea el pipeline e imprime el resultado de una frase
```

4.2 Reconocimiento (NER)

Identifica y clasifica nombres de personas, organizaciones o ubicaciones en un texto.

Tarea: "ner"

Parámetros Extra: aggregation_strategy="simple"

Modelo Sugerido: "mrms488/bert-spanish-cased-finetuned-ner"

```
from transformers import pipeline

# TODO: Crea el pipeline con los parámetros sugeridos y pásale
```

🔗 4.3 Preguntas y Respuestas (QA)

Extrae la respuesta exacta a una pregunta basándose en un texto de contexto.

Tarea: "question-answering"

Uso: qa(question="...", context="...")

```
from transformers import pipeline

# TODO: Construye la lógica del pipeline QA
```

🗣️ 4.5 Texto a Voz (TTS)

Sintetiza texto escrito convirtiéndolo en audio hablado generado por IA.

Tarea: "text-to-speech"

Modelo Sugerido: "facebook/mms-tts-spa" (Para español)

```
from transformers import pipeline
import soundfile as sf

# TODO: Crea el pipeline TTS
# TODO: Genera el audio a partir de un texto
# TODO: Usa sf.write("nombre.wav", audio["audio"], samplerate=audio["sampling_rate"]) para guardar
```

🗨️ 4.4 Traducción Automática

Convierte texto de un idioma a otro de forma automática.

Tarea: "translation_en_to_fr"

Modelo Sugerido: "Helsinki-NLP/opus-mt-en-fr"

```
from transformers import pipeline

# TODO: Crea el traductor y traduce una frase del inglés al fr
```